

FREE RESOURCE

Claude Code CMS Dashboard Playbook

Build the safe client handoff layer for AI-built websites.

In simple terms: the CMS is the control panel that lets you or your client edit a live website - text, pages, images, SEO, and publishing - without opening the code or touching your private developer account.

1. What this CMS is all about

Claude Code can build the website, but the CMS gives the owner or client a safe editing layer after the site is deployed.

- The website stays professional and protected.
- The client can update copy, pricing, testimonials, images, SEO settings, and pages.
- You do not need to manually edit every small change.
- Every publish can be saved as a version or snapshot.
- One master CMS can manage many client websites.

2. The problem it solves

Without a CMS, clients ask you for every tiny update. Giving them your developer login is risky, and giving them raw code access is worse.

- A plain AI-coded website is hard to hand off because the client still needs a simple editor.
- The CMS turns a one-time website build into a more professional offer.
- The same system can support recurring hosting, maintenance, analytics, and support revenue.

3. What the CMS should allow

Feature	Purpose
Page editor	Edit headlines, sections, copy, buttons, images, testimonials, and simple spacing.
SEO panel	Set page title, meta description, target keyword, and SEO checklist status.
Multi-page manager	Add or edit Home, About, Pricing, Blog, Article, or landing pages.
Preview modes	Check desktop, tablet, and mobile before publishing.
AI edit chat	Ask AI to rewrite selected copy or suggest improvements.
Publish button	Changes stay drafted until the user intentionally publishes.
Version history	Every publish creates a snapshot for rollback.
Client access	Each client only edits the website they are allowed to access.
Form inbox	Optional: feed website leads into the CMS.

4. Simple architecture

Think of the system as two connected products: the public website and the private editing dashboard.

- Client / You -> CMS Dashboard -> edits content
- CMS Dashboard -> MongoDB Atlas -> saves content, SEO fields, settings, and versions
- CMS Dashboard -> OpenRouter / AI API -> rewrites selected copy
- CMS Dashboard -> Vercel Website -> publishes approved changes
- GitHub Repo -> stores the code privately

Recommended stack

- Claude Code
- Next.js or React dashboard
- MongoDB Atlas
- Vercel
- GitHub
- OpenRouter or another AI API
- Password or login system

5. Implementation plan

Step 1 - Build the website first

Use Claude Code to create a clean responsive website from a design reference. Deploy it to Vercel and store the code in a private GitHub repo.

Step 2 - Identify editable content

Mark what the client can safely edit: text, pricing, testimonials, images, SEO fields, blog posts, and pages. Do not expose core layout or code in version one.

Step 3 - Build the CMS dashboard

Create a separate admin dashboard that lists all websites, pages, and editable sections. Add preview, edit, save draft, and publish actions.

Step 4 - Connect MongoDB Atlas

Store page content as JSON-like documents: siteId, page slug, sections, SEO settings, status, createdAt, updatedAt, and versions.

Step 5 - Add AI editing

Connect OpenRouter or another AI API so users can ask AI to rewrite, shorten, improve, or optimize selected copy.

Step 6 - Add safety rules

Validate every change before publishing. Allow copy, images, and page fields, but block dangerous code edits unless you intentionally build an advanced mode.

Step 7 - Add version history

Every publish should create a snapshot so you can restore a previous version if the client makes a mistake.

Step 8 - Deploy and test

Deploy the CMS to its own Vercel URL. Test edit, publish, refresh, mobile preview, rollback, and login permissions.

Step 9 - Hand off to the client

Give the client the editor link, password or login, and a short usage guide. Keep GitHub, Vercel, and MongoDB master access under your control.

6. MongoDB data model example

```
Site
- siteId
- clientName
- liveUrl
- vercelProjectId
- editorPasswordHash

Page
- pageId
- siteId
- slug: /, /about, /pricing
- title
- seoTitle
- metaDescription
- sections: [{ id, type, content, imageUrl, buttonText, buttonUrl }]
- status: draft or published

Version
- versionId
- pageId
- publishedAt
- publishedBy
- snapshot
- notes
```

7. Claude Code prompt you can use

Build a CMS dashboard for my deployed website.

Goal:

I want one admin dashboard where I can manage multiple client websites. Each website should have editable pages, SEO settings, preview, save draft, publish, and version history.

Tech stack:

- Next.js or React for the dashboard
- MongoDB Atlas for content and version storage
- Vercel for deployment
- GitHub repo is already connected
- OpenRouter API for AI-assisted copy editing

Rules:

- Clients can edit copy, images, buttons, SEO fields, and add simple pages.
- Clients cannot edit raw code or break the core design.
- Every change must be validated before publishing.
- Every publish creates a version snapshot for rollback.
- Each client only sees their own website after login/password.

Build this in phases:

1. Data model
2. Dashboard UI
3. Page editor
4. Preview and publish flow
5. MongoDB connection
6. AI rewrite tool
7. Login/access control
8. Deployment instructions

8. How to package this for clients

- Start with a simple CMS first: edit text, images, SEO, and pages.
- Do not build a full code editor in version one because it increases risk.
- Position it as: "Your client can edit the website, but they cannot destroy the design."
- Use it as an agency or white-label website system: build sites fast, hand off safe editing, and charge for setup plus optional hosting/support.
- Later upgrades can include blog publishing, lead form inbox, analytics, templates, client billing, and white-label branding.

9. Final checklist before giving it to clients

- Live website deployed and working
- CMS deployed on its own URL
- MongoDB connected and tested
- AI editing API connected and budget-limited
- Client login/password works
- Client can only access their own site
- Publish flow updates the live site correctly
- Version rollback works
- SEO panel saves correctly
- Mobile preview checked
- Client handoff guide created

Ready to go deeper?

This playbook is the start. In Vibe Coding Mastery, you learn how to turn Claude Code workflows into deployed products, client systems, and repeatable AI builds.

Join the Vibe Coding Build -> <https://builds.digicuratoragency.com/>